SCHOOL OF COMPUTER SCIENCES
UNIVERSITI SAINS MALAYSIA

CPC452 Animation & Virtual Reality

Semester II, Academic Session 2025/2026

Assignment 2 Report

Lecturer: Associate Professor Dr. Ahmad Sufril Azlan Mohamed

Group Number: 6

Name	Matric Number	USM Email
Lai Yicheng	163729	laiyicheng@student.usm.my
Koay Chun Keat	164592	koayck@student.usm.my
Yong Pei Yi, Angeline	163842	angelineyong@student.usm.my
Tan Ke Jun	163204	tankejun219@student.usm.my
Tan Yi Pin	163382	yipintan323@gmail.com

Submission Date: 26 June 2026

Abstract

This project evaluates MongoDB Community 8.0 in two deployment configurations — a stand-alone local installation on Ubuntu Linux and a Docker-containerized deployment — using a credit card fraud detection dataset of approximately 480 MB. The dataset, sourced from Kaggle, contains approximately 1.85 million transaction records across 22 fields, of which only 9,651 (0.52%) are fraudulent. Four meaningful aggregation pipeline queries were designed and executed identically on both platforms to analyse fraud patterns by transaction category, merchant, time of day, and statistical comparison between fraudulent and legitimate transactions. The analysis shows that fraud is concentrated in the `grocery_pos` and `shopping_net` categories and that fraudulent transactions average $530.66 — roughly 7.8 times the $67.65 average of legitimate transactions — making transaction amount a strong fraud signal. Benchmarking across five runs per query shows that Docker is approximately 15–19% slower than the stand-alone installation on computation-heavy queries, while delivering comparable performance for lightweight, index-covered queries. The stand-alone deployment provides direct hardware access and a simpler query interaction workflow, whereas Docker offers superior portability and reproducibility with minimal overhead on simple queries. Both implementations produce identical query results, confirming correctness across deployments. These findings highlight the practical trade-offs in MongoDB deployment strategy for big data analytics workloads.

Contents

1 Introduction

The rapid growth of digital financial services has generated enormous volumes of transaction data that must be processed, stored, and analysed efficiently. Traditional relational databases, designed around fixed schemas and normalised tables, often struggle to adapt to the heterogeneous structure of modern datasets. MongoDB, a document-oriented NoSQL database, addresses these limitations through flexible BSON document storage, a rich aggregation pipeline, and native horizontal scaling capabilities [4]. Its ability to store nested and varied records makes it well-suited for datasets that combine categorical, numerical, and temporal fields in a single document [2].

Alongside the evolution of database software, containerisation technology has fundamentally changed how applications are deployed and managed. Docker packages a service together with all its runtime dependencies into a portable, isolated container that runs consistently across different host environments [1]. This portability is particularly valuable for database deployments where environment differences can cause subtle behavioural discrepancies. However, containerisation introduces an additional filesystem abstraction layer (the Overlay2 storage driver on Linux) that can affect I/O-intensive workloads compared to a direct stand-alone installation.

This project examines both deployment modes in a fraud analytics context. By loading the same dataset, running the same aggregation pipelines on both platforms, and measuring execution time objectively, the project produces a reproducible, evidence-based comparison. The three dimensions evaluated are ease of use during setup and operation, query construction and execution workflow, and data processing speed under equivalent query logic.

1.1 Objectives

The objectives of this project are:

- To install and configure MongoDB Community 8.0 in both a stand-alone environment (Ubuntu Linux) and a Docker-containerised environment.
- To import a medium-scale credit card fraud detection dataset (˜480 MB) into both

MongoDB deployments using the same collection schema.

- To design and execute four meaningful aggregation pipeline queries that provide analytical insights about fraud patterns in the dataset.
- To benchmark each query across five runs on both platforms and compare average execution times.
- To compare the two deployments on ease of use, query creation workflow, and data processing speed.
- To derive practical lessons from the deployment and analysis experience for future MongoDB use.

2 Project Content

2.1 Dataset Description

The dataset used in this project is the *Credit Card Transactions Fraud Detection Dataset* published by Kartik Shenoy on Kaggle [5]. It is released under the CC0 Public Domain licence and was generated synthetically using the Sparkov Data Generation tool to simulate realistic credit card transaction behaviour without exposing real cardholder data. The synthetic nature of the data makes it freely reproducible and suitable for academic benchmarking.

The dataset is distributed as two CSV files. Table 1 summarises their sizes and record counts.

Table 1: Dataset files and record counts

File	**Approximate Size**	**Approximate Records**
`fraudTrain.csv`	335 MB	1,296,675
`fraudTest.csv`	144 MB	555,719
Combined total	**≈480 MB**	**≈1,852,394**

Both files are imported into a single MongoDB collection named `transactions` inside the

`cpc451` database so that all queries operate on the combined dataset. Table 2 describes the fields used in the four analytical queries.

Table 2: Key fields in the fraud detection dataset used in queries

Field	**Type**	**Description**
`trans_date_trans_time`	string	Transaction datetime in `"YYYY-MM-DD HH:MM:SS"` format
`merchant`	string	Name of the merchant where the transaction occurred
`category`	string	Transaction category (e.g. `grocery_pos`, `shopping_net`, `misc_net`)
`amt`	float	Transaction amount in USD
`trans_num`	string	Unique transaction identifier (UUID)
`is_fraud`	int	Fraud label: `1` = fraudulent, `0` = legitimate

The dataset was selected for three reasons. First, fraud detection is a meaningful real-world analytics domain where query results provide actionable insights. Second, the combined size of approximately 480 MB satisfies the project requirement of 400–500 MB and generates a collection large enough to produce measurable execution time differences between the two platforms. Third, the schema contains categorical (`category`, `merchant`), numerical (`amt`), temporal (`trans_date_trans_time`), and label (`is_fraud`) fields, enabling diverse aggregation queries that go beyond simple CRUD operations.

2.2 Selection of MongoDB Implementation Platform

This project evaluates MongoDB in two deployment modes: a **stand-alone installation** directly on the host operating system and a **Docker-containerised deployment**. These two modes represent the practical spectrum between traditional server-based and modern containerised database operation.

Stand-alone MongoDB installs the MongoDB Community Server directly on the host

machine via the OS package manager (`apt` on Ubuntu). The `mongod` process runs as a system service with direct access to the host filesystem and CPU. This mode is typical in single-machine development and production environments where the operator has full control over the operating system.

Docker-containerised MongoDB runs the official `mongo:8.0` image managed by Docker Compose. The container exposes MongoDB on host port 27018 (mapped from container port 27017) to avoid conflict with the stand-alone instance on port 27017. Data is persisted on a named Docker volume (`cpc451_mongo_data`) and the CSV input files are mounted into the container as a read-only volume at `/data/input`. This mode is typical in cloud-native, microservice, and reproducible research environments.

Both implementations use MongoDB Community 8.0, the same dataset, the same collection schema, and the same aggregation pipeline logic. This ensures that observed performance differences reflect deployment overhead rather than query or data differences. Table 3 summarises the key configuration differences.

Table 3: Configuration comparison: stand-alone vs. Docker

Aspect	**Stand-alone**	**Docker**
MongoDB version	8.0 (apt package)	8.0 (`mongo:8.0` image)
Host port	27017	27018
Data storage	Host filesystem	Named Docker volume (`cpc451_mongo_data`)
Input files	Host path	Read-only container volume mount
Process management	`systemctl` service	`docker compose`
Query execution	Direct `mongosh`	`docker exec ... mongosh`

2.3 Installation, Configuration, Database Construction, and Data Entry

2.3.1 Stand-alone MongoDB

Installation. MongoDB Community 8.0 is installed on Ubuntu 22.04 (or compatible Debian-based Linux) using the official MongoDB `apt` repository [3]. The installation script (`code/standalone/setup.sh`) performs five steps automatically:

Listing 1: Stand-alone MongoDB installation steps (setup.sh)

```
# Step 1: Import MongoDB GPG signing key
curl -fsSL https://www.mongodb.org/static/pgp/server-8.0.asc \
  | sudo gpg --dearmor -o /usr/share/keyrings/mongodb-server-8.0.gpg

# Step 2: Add the official MongoDB apt repository
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ] \
https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/8.0 multiverse" \
  | sudo tee /etc/apt/sources.list.d/mongodb-org-8.0.list

# Step 3: Install the mongodb-org package
sudo apt-get update -q && sudo apt-get install -y mongodb-org

# Step 4: Enable and start the mongod system service
sudo systemctl daemon-reload
sudo systemctl enable mongod
sudo systemctl start mongod

# Step 5: Verify installation
mongosh --version && mongoimport --version
```

After setup, `mongod` listens on `localhost:27017` and starts automatically on system boot. A Windows equivalent (`setup.ps1`) is also provided, which downloads the MongoDB MSI installer, installs Database Tools separately, and starts the service.

Data import. Once the service is running, both CSV files are imported into the `cpc451.transactions` collection using `mongoimport`. The `--headerline` flag reads field names from the first row of each CSV and `--numInsertionWorkers 4` enables parallel

insertion to reduce import time.

Listing 2: Data import for stand-alone deployment (import.sh)

```
DB="cpc451"
COL="transactions"

# Drop any existing collection for a clean import
mongosh --quiet --eval "db.${COL}.drop()" "$DB"

# Import training set (approx. 1.3 M records)
mongoimport --db "$DB" --collection "$COL" \
  --type csv --headerline --numInsertionWorkers 4 \
  --file code/input/fraudTrain.csv

# Import test set (approx. 556 K records) -- appended to same collection
mongoimport --db "$DB" --collection "$COL" \
  --type csv --headerline --numInsertionWorkers 4 \
  --file code/input/fraudTest.csv
```

After import, the total document count is verified:

```
mongosh --quiet --eval "
  const total = db.transactions.countDocuments();
  const fraud = db.transactions.countDocuments({ is_fraud: 1 });
  print('Total: ' + total);
  print('Fraud: ' + fraud);
  print('Legit: ' + (total - fraud));
" cpc451
```

This confirms the imported collection contains all 1,852,394 records and reveals the strong class imbalance of the dataset:

```
Total: 1852394
Fraud:    9651
Legit: 1842743
```

Index creation. Four compound indexes are created on the `transactions` collection before benchmarking. Each index is designed to match the leading match field (`is_fraud`)

and the grouping or projection field used by the corresponding query, enabling MongoDB to satisfy the `$match` stage with an index scan rather than a collection scan.

Listing 3: Index creation script (create_indexes.js)

```
// Q1: fraud count by category
db.transactions.createIndex({ is_fraud: 1, category: 1 },
  { name: "idx_fraud_category" });

// Q2: fraud count by merchant
db.transactions.createIndex({ is_fraud: 1, merchant: 1 },
  { name: "idx_fraud_merchant" });

// Q3 & Q4: fraud amount statistics
db.transactions.createIndex({ is_fraud: 1, amt: 1 },
  { name: "idx_fraud_amt" });
```

These indexes are applied by running:

```
mongosh cpc451 code/shared/create_indexes.js
```

2.3.2 Container MongoDB (Docker)

Installation. The Docker deployment uses `docker-compose.yml` to provision a `mongo:8.0` container named `mongodb_cpc451`. No OS-level MongoDB installation is required on the host; only Docker Engine and Docker Compose must be installed.

Listing 4: Docker Compose configuration (docker-compose.yml)

```
services:
  mongodb:
    image: mongo:8.0
    container_name: mongodb_cpc451
    restart: unless-stopped
    ports:
      - "27018:27017"   # avoid conflict with standalone on 27017
    volumes:
      - mongo-data:/data/db
      - ../input:/data/input:ro          # CSV files (read-only)
      - ../shared/create_indexes.js:/scripts/create_indexes.js:ro
```

```
      - ../shared/benchmark.js:/scripts/benchmark.js:ro
      - ../shared/queries.js:/scripts/queries.js:ro

volumes:
  mongo-data:
    name: cpc451_mongo_data
```

The container is started with a single command:

```
docker compose up -d
```

The input CSVs mounted at `/data/input` inside the container and the shared query scripts mounted at `/scripts` are available immediately after startup without any additional file copying.

Data import. Data import follows the same logic as the stand-alone deployment, but all commands run inside the container via `docker exec`:

Listing 5: Data import for Docker deployment (docker/import.sh)

```
CONTAINER="mongodb_cpc451"
DB="cpc451"
COL="transactions"

docker exec "$CONTAINER" \
  mongosh --quiet --eval "db.${COL}.drop()" "$DB"

docker exec "$CONTAINER" \
  mongoimport --db "$DB" --collection "$COL" \
    --type csv --headerline --numInsertionWorkers 4 \
    --file "/data/input/fraudTrain.csv"

docker exec "$CONTAINER" \
  mongoimport --db "$DB" --collection "$COL" \
    --type csv --headerline --numInsertionWorkers 4 \
    --file "/data/input/fraudTest.csv"
```

Index creation. The shared `create_indexes.js` script is mounted into the container and executed in the same way as the stand-alone deployment:

```
docker exec mongodb_cpc451 \
  mongosh cpc451 /scripts/create_indexes.js
```

2.4 Meaningful Queries in MongoDB (Stand-alone)

All four queries are aggregation pipeline queries targeting the `cpc451.transactions` collection. They are executed in the stand-alone environment using:

```
mongosh cpc451 code/shared/queries.js
```

2.4.1 Query 1: Fraud Count by Transaction Category

Purpose. This query answers: *Which transaction category has the highest number of fraudulent transactions?* It helps analysts identify the business segment most exploited by fraudsters, which can inform targeted fraud prevention policies.

Pipeline stages.

1. `$match {is_fraud: 1}` — filters to fraudulent transactions only, using the `idx_fraud_category` index.
2. `$group {_id: "$category"}` — groups documents by transaction category and counts occurrences with `$sum: 1`.
3. `$sort {fraud_count: -1}` — orders results from highest to lowest fraud count.

Listing 6: Q1 — Fraud count by transaction category

```
db.transactions.aggregate([
  { $match: { is_fraud: 1 } },
  { $group: { _id: "$category", fraud_count: { $sum: 1 } } },
  { $sort: { fraud_count: -1 } }
])
```

Result. Table 4 lists the fraud count for all 14 categories returned by the query (identical on both platforms).

Table 4: Q1 result — fraudulent transaction count by category

Category	Count	Category	Count
grocery_pos	2,228	kids_pets	304
shopping_net	2,219	entertainment	292
misc_net	1,182	personal_care	290
shopping_pos	1,056	home	265
gas_transport	772	food_dining	205
misc_pos	322	health_fitness	185
		grocery_net	175
		travel	156

Insight. The output reveals which merchant categories are disproportionately affected by fraud. Point-of-sale grocery purchases (`grocery_pos`, 2,228 cases) and online shopping (`shopping_net`, 2,219 cases) dominate, together accounting for roughly 46% of all fraudulent transactions, followed by `misc_net` and `shopping_pos`. The prominence of online (`_net`) categories is consistent with card-not-present fraud being easier to perpetrate than in-person fraud. This information can be used to apply tighter authentication thresholds on high-risk categories such as online shopping and grocery point-of-sale.

2.4.2 Query 2: Top 10 Merchants by Fraud Count

Purpose. This query answers: *Which merchants are most frequently involved in fraudulent transactions?* Identifying merchants with consistently high fraud counts supports merchant-level risk scoring and can trigger enhanced monitoring.

Pipeline stages.

1. `$match {is_fraud: 1}` — filters to fraudulent transactions, using the `idx_fraud_merchant` index.
2. `$group {_id: "$merchant"}` — groups by merchant name and counts fraudulent transactions.

3. `$sort {fraud_count: -1}` — orders by descending fraud count.
4. `$limit 10` — returns only the top 10 merchants.

Listing 7: Q2 — Top 10 merchants by fraud count

```
db.transactions.aggregate([
  { $match: { is_fraud: 1 } },
  { $group: { _id: "$merchant", fraud_count: { $sum: 1 } } },
  { $sort: { fraud_count: -1 } },
  { $limit: 10 }
])
```

Result. The top three merchants by fraud count are `fraud_Kilback LLC` (62 fraudulent transactions), `fraud_Rau and Sons` (60), and `fraud_Kozey-Boehm` (60), with the remainder of the top 10 ranging from 53 to 57.

Insight. The result shows specific merchant identifiers with the highest concentration of fraud, enabling prioritised fraud investigation. Notably, the counts across the top 10 merchants are tightly clustered (53–62 cases), which indicates that fraud is spread thinly across many merchants rather than dominated by a few outliers — no single merchant accounts for more than 0.65% of the 9,651 fraudulent transactions. Merchant identity is therefore a weaker standalone fraud signal than transaction category or amount. The `$limit` stage reduces output to the most actionable subset, making it practical for operational dashboards.

2.4.3 Query 3: Average Fraudulent Transaction Amount by Hour of Day

Purpose. This query answers: *At what hour of the day do the largest fraudulent transactions tend to occur?* Time-of-day analysis helps model temporal fraud risk and can inform real-time scoring models that adjust thresholds depending on the hour.

Pipeline stages.

1. `$match {is_fraud: 1}` — filters to fraudulent transactions.
2. `$addFields {hour}` — extracts the two-digit hour from the datetime string using `$substr` (characters at positions 11–12) and converts it to an integer with `$toInt`.

3. `$group {_id: "$hour"}` — groups by hour and computes average amount (`$avg`) and count (`$sum`).
4. `$sort {_id: 1}` — orders results chronologically from hour 0 to hour 23.

Listing 8: Q3 — Average fraudulent amount by hour of day

```
db.transactions.aggregate([
  { $match: { is_fraud: 1 } },
  { $addFields: {
      hour: { $toInt: { $substr: ["$trans_date_trans_time", 11, 2] } }
  }},
  { $group: {
      _id: "$hour",
      avg_amount: { $avg: "$amt" },
      tx_count:   { $sum: 1 }
  }},
  { $sort: { _id: 1 } }
])
```

Result. The query exposes two distinct temporal patterns. By *volume*, fraud is heavily concentrated in the late-night window: hours 22:00 and 23:00 alone contain 2,481 and 2,442 fraudulent transactions respectively (together more than half of all fraud), with a secondary cluster between midnight and 03:00 (roughly 800 cases per hour). By *average amount*, however, the peak occurs in the afternoon and evening, reaching $709 at 14:00 and staying elevated ($597–$687) through to 23:00, whereas the high-volume midnight hours carry markedly lower averages of around $345–$360.

Insight. Contrary to the intuition that fraud value would peak overnight, the data shows that fraudsters execute many *small-value* transactions in the late-night and early-morning hours but reserve their *high-value* fraud for busy daytime and evening periods, when large amounts blend in more easily with the surge of legitimate spending. A real-time scoring model could exploit both signals: flag the elevated *frequency* of fraud after 22:00 and the elevated *average value* of fraud during the 12:00–22:00 window. The `$addFields` and `$substr` approach extracts temporal features directly from the stored string without requiring a date parsing library, demonstrating MongoDB's expressive pipeline capabilities.

2.4.4 Query 4: Statistical Comparison Between Fraud and Legitimate Transactions

Purpose. This query answers: *How do fraudulent and legitimate transactions differ statistically in amount?* It computes count, average, maximum, minimum, and total transaction amounts for both classes simultaneously in a single aggregation pass.

Pipeline stages.

1. `$facet` — runs two independent sub-pipelines in parallel within the same aggregation.
2. `fraud` branch — matches {`is_fraud: 1`} and computes five statistical measures.
3. `legitimate` branch — matches {`is_fraud: 0`} and computes the same five measures.

Listing 9: Q4 — Statistical comparison: fraud vs. legitimate transactions

```
db.transactions.aggregate([
  { $facet: {
      fraud: [
        { $match: { is_fraud: 1 } },
        { $group: {
            _id: null,
            count:     { $sum: 1 },
            avg_amt:   { $avg: "$amt" },
            max_amt:   { $max: "$amt" },
            min_amt:   { $min: "$amt" },
            total_amt: { $sum: "$amt" }
        }}
      ],
      legitimate: [
        { $match: { is_fraud: 0 } },
        { $group: {
            _id: null,
            count:     { $sum: 1 },
            avg_amt:   { $avg: "$amt" },
            max_amt:   { $max: "$amt" },
            min_amt:   { $min: "$amt" },
```

```
            total_amt: { $sum: "$amt" }
        }}
      ]
  }}
])
```

Result. Table 5 reports the five statistics computed for each class (identical on both platforms).

Table 5: Q4 result — statistical comparison of fraudulent vs. legitimate transaction amounts (USD)

Statistic	**Fraudulent**	**Legitimate**
Transaction count	9,651	1,842,743
Average amount	530.66	67.65
Minimum amount	1.06	1.00
Maximum amount	1,376.04	28,948.90
Total amount	5,121,413.29	124,663,918.72

Insight. The `$facet` stage is the most computationally expensive among the four queries because it effectively performs two full collection scans in a single pipeline execution, which explains its significantly longer execution time (over 2 seconds). The results expose a strong amount-based fraud signal: fraudulent transactions average \$530.66, roughly 7.8 times the \$67.65 average of legitimate transactions, despite fraud representing only 0.52% of all records. Equally revealing is the *ceiling* effect — the largest fraudulent transaction (\$1,376.04) is far below the largest legitimate one (\$28,948.90), suggesting that fraudsters deliberately keep amounts within a mid-value band (roughly \$1–\$1,400) to avoid the manual-review thresholds that very large transactions attract. This bounded yet elevated-average profile makes transaction amount one of the most discriminative features for amount-based fraud-detection rules.

2.5 Meaningful Queries in MongoDB (Container)

The same four queries are executed on the Docker-containerised MongoDB instance. The aggregation pipeline logic, field names, and output structure are identical to the stand-alone version. The only difference is the execution mechanism: all `mongosh` commands are issued through `docker exec` rather than directly on the host shell.

The shared query script is mounted into the container at `/scripts/queries.js` via the Docker volume configuration in `docker-compose.yml`. Queries are executed with:

```
docker exec mongodb_cpc451 \
  mongosh cpc451 /scripts/queries.js
```

The four queries are listed below with their execution commands. The pipeline definitions are identical to those in Section 2.4 and are not repeated. Executing them inside the container produced output values byte-for-byte identical to the stand-alone results reported in Section 2.4 (for example, Q4 returned the same fraudulent average of $530.66 and Q1 the same ranking led by `grocery_pos`); the per-query result values are therefore not duplicated here.

2.5.1 Query 1: Fraud Count by Transaction Category

Listing 10: Executing Q1 in the Docker container

```
docker exec mongodb_cpc451 mongosh --quiet cpc451 --eval "
db.transactions.aggregate([
  { \$match: { is_fraud: 1 } },
  { \$group: { _id: '\$category', fraud_count: { \$sum: 1 } } },
  { \$sort: { fraud_count: -1 } }
]).forEach(printjson)"
```

2.5.2 Query 2: Top 10 Merchants by Fraud Count

Listing 11: Executing Q2 in the Docker container

```
docker exec mongodb_cpc451 mongosh --quiet cpc451 --eval "
db.transactions.aggregate([
  { \$match: { is_fraud: 1 } },
```

```
  { \$group: { _id: '\$merchant', fraud_count: { \$sum: 1 } } },
  { \$sort: { fraud_count: -1 } },
  { \$limit: 10 }
]).forEach(printjson)"
```

2.5.3 Query 3: Average Fraudulent Amount by Hour of Day

Listing 12: Executing Q3 in the Docker container

```
docker exec mongodb_cpc451 mongosh --quiet cpc451 --eval "
db.transactions.aggregate([
  { \$match: { is_fraud: 1 } },
  { \$addFields: {
      hour: { \$toInt: { \$substr: ['\$trans_date_trans_time', 11, 2] } }
  }},
  { \$group: {
      _id: '\$hour',
      avg_amount: { \$avg: '\$amt' },
      tx_count: { \$sum: 1 }
  }},
  { \$sort: { _id: 1 } }
]).forEach(printjson)"
```

2.5.4 Query 4: Statistical Comparison Between Fraud and Legitimate Transactions

Listing 13: Executing Q4 in the Docker container

```
docker exec mongodb_cpc451 mongosh --quiet cpc451 --eval "
db.transactions.aggregate([
  { \$facet: {
      fraud: [
        { \$match: { is_fraud: 1 } },
        { \$group: { _id: null, count: { \$sum: 1 },
            avg_amt: { \$avg: '\$amt' }, max_amt: { \$max: '\$amt' },
            min_amt: { \$min: '\$amt' }, total_amt: { \$sum: '\$amt' } } }
      ],
      legitimate: [
```

```
        { \$match: { is_fraud: 0 } },
        { \$group: { _id: null, count: { \$sum: 1 },
            avg_amt: { \$avg: '\$amt' }, max_amt: { \$max: '\$amt' },
            min_amt: { \$min: '\$amt' }, total_amt: { \$sum: '\$amt' } } }
      ]
  }}
]).forEach(printjson)"
```

2.6 Comparison, Discussion, and Observation

2.6.1 Ease of Use

Table 6 summarises the practical differences in setup, operation, and daily use between the two implementations.

Table 6: Ease of use comparison

Aspect	Stand-alone	Docker
Installation effort	Requires OS-level package manager steps, GPG key, and `apt` repository setup. Elevated (`sudo`) access required.	Single `docker compose up -d` command. No OS package changes.
Cross-platform support	Script must be adapted per OS (separate `setup.sh` for Linux, `setup.ps1` for Windows).	Identical `docker-compose.yml` works on any OS with Docker installed.
Service management	Managed via `systemctl`; persists across reboots automatically.	Managed via `docker compose`; container restarts on daemon restart (`restart: unless-stopped`).
Query execution	Direct: `mongosh cpc451 script.js`	Indirect: `docker exec mongodb_cpc451 mongosh ...`
Data access	Host paths used directly in commands.	CSV files must be volume-mounted into the container before use.
Reproducibility	Depends on host OS version and package availability.	Fully reproducible: container image version is pinned in `docker-compose.yml`.

2.6.2 Query Creation Workflow

Both platforms use the same `mongosh` JavaScript shell to write and test queries. The aggregation pipeline syntax, operators, and output format are identical. The primary workflow difference is the execution prefix: stand-alone queries are run directly from the shell, while Docker queries require `docker exec` as a wrapper.

For iterative development, the stand-alone environment is marginally more convenient because query scripts can be edited on the host and executed immediately without restarting or re-mounting containers. The Docker environment requires either re-mounting the shared script volume or copying updated files into the container. However, once the shared volume mounts are configured in `docker-compose.yml`, the shared scripts (`queries.js`, `benchmark.js`, `create_indexes.js`) are accessible inside the container at the paths defined in the Compose file, which eliminates repeated file copying.

2.6.3 Data Processing Speed

Each of the four queries was executed five times on both platforms after indexes were created. Table 7 presents the individual run times and the 5-run average for each query and implementation.

Table 7: Query execution times (ms) — 5 runs per query per implementation

Query	**Impl.**	**Run 1**	**Run 2**	**Run 3**	**Run 4**	**Run 5**	**Avg (ms)**
Q1	Standalone	18	5	4	4	4	**7.0**
Q1	Docker	7	3	3	3	4	**4.0**
Q2	Standalone	6	4	5	5	4	**4.8**
Q2	Docker	4	5	7	5	6	**5.4**
Q3	Standalone	26	24	26	27	27	**26.0**
Q3	Docker	29	30	34	31	30	**30.8**
Q4	Standalone	3051	2427	2342	2061	2053	**2386.8**
Q4	Docker	3205	2752	2321	2606	2796	**2736.0**

Discussion. Q1 and Q2 are lightweight queries that hit compound indexes efficiently, returning results in single-digit milliseconds on both platforms. For Q1, the stand-alone first run (18 ms) is noticeably higher than subsequent runs (4–5 ms), reflecting the cold-cache startup cost of loading index pages and the mongosh JavaScript runtime into memory.

Docker shows a similar but smaller first-run spike (7 ms). Across warm runs (runs 2–5), both platforms are essentially equivalent for these simple queries.

Q3 introduces a `$addFields` stage that extracts the hour digit from a stored string, adding CPU work beyond pure index lookups. Stand-alone averages 26.0 ms while Docker averages 30.8 ms — an 18.5% difference. The overhead in the Docker case is attributable to the Overlay2 filesystem driver and the additional network stack layer between the container and the host, which add latency even for operations that are primarily CPU-bound.

Q4 is the most demanding query because the `$facet` stage runs two full-collection traversals in a single pipeline. Stand-alone averages 2386.8 ms while Docker averages 2736.0 ms — a 14.6% difference. The larger absolute overhead (349 ms) compared to Q3 reflects that Docker's I/O overhead scales with the volume of data scanned, since scan-heavy pipelines read far more pages through the Overlay2 storage layer than index-covered queries do. Systematic performance evaluation of document-oriented databases under varying operation profiles has likewise shown that storage and replication overhead is workload-dependent rather than a constant factor [6].

The stand-alone implementation is consistently faster for computation-heavy queries (Q3 and Q4), while both platforms perform comparably on index-covered lightweight queries (Q1 and Q2). Table 8 provides the head-to-head summary across all three comparison dimensions.

Table 8: Overall comparison summary

Dimension	**Stand-alone**	**Docker**
Ease of use	More installation steps; OS-specific scripts; direct `mongosh` access	Simpler startup; fully portable; extra `docker exec` prefix for queries
Query creation	Direct shell interaction; easiest for iterative development	Requires volume mounts or `docker cp` to update scripts
Speed (lightweight Q1–Q2)	≈5–7 ms average	≈4–5 ms average (comparable)
Speed (complex Q3–Q4)	26 ms and 2387 ms	31 ms and 2736 ms (˜15–19% slower)
Reproducibility	OS and version dependent	Fully reproducible via pinned image

2.6.4 Observations

Several practical observations were made during the implementation and benchmarking process.

First, the first benchmark run consistently shows higher execution time than subsequent runs on both platforms. This is expected behaviour: the first run loads index pages and the aggregation runtime into the OS page cache, while subsequent runs benefit from warm memory. For production use, the warm-cache steady-state time is the more representative metric.

Second, the Docker container runs MongoDB as UID 999 by default. The input directory mounted into the container must be readable by this UID; on some Linux systems, this requires explicit file permission adjustment (`chmod o+r`) before mounting. The stand-alone deployment does not have this constraint.

Third, running both implementations simultaneously on the same machine is possible because the Docker instance uses host port 27018 while the stand-alone instance uses

27017. This avoids port conflict and simplifies comparative testing without needing two separate machines.

Fourth, query result correctness was verified by running the same query on both platforms and comparing outputs. Both implementations produced identical results for all four queries, confirming that the Docker containerisation layer does not affect query semantics or aggregation correctness.

2.7 Concluding Remarks

Both MongoDB implementations successfully fulfil the project requirements. The standalone deployment is appropriate when maximising query performance is the primary concern and the operator has control over the host operating system. The Docker deployment is preferable when portability, reproducibility, and simplified provisioning are more important than the marginal performance advantage of direct hardware access. For the fraud detection use case evaluated here, both platforms can handle the full 1.85 million document dataset with sub-second response for three of the four queries, demonstrating that MongoDB is a capable choice for medium-scale analytical workloads regardless of deployment mode.

3 Lesson Learnt

This project produced several meaningful lessons that extend beyond the technical results.

Index design is the single most impactful factor in query performance. Before indexes were created, Q1 and Q2 required full collection scans across 1.85 million documents, taking several seconds. After creating compound indexes on {`is_fraud, category`} and {`is_fraud, merchant`}, the same queries returned in under 10 ms. This reinforced that query optimisation in MongoDB begins at the schema and index design stage, not at the query syntax level.

The aggregation pipeline is a powerful tool for analytical queries. All four queries used multi-stage aggregation pipelines combining `$match`, `$group`, `$sort`, `$addFields`, `$limit`, and `$facet`. Writing these pipelines clarified how MongoDB separates data filter-

ing from data transformation and how stages can be chained to express complex analytical logic without leaving the database layer. This is a practical advantage over relational databases where equivalent operations would require multiple joins and subqueries.

Docker simplifies environment management but requires careful volume configuration. Setting up the stand-alone installation required OS-level steps that differed between Ubuntu Linux and Windows. Docker removed this complexity by encapsulating MongoDB in a container. However, correctly mapping host directories as container volumes and handling file permissions for the MongoDB UID required careful configuration. The lesson is that Docker shifts complexity from installation to configuration management.

Performance benchmarking requires controlled conditions. Early benchmark runs showed high variance because both the stand-alone and Docker instances were running simultaneously on the same machine, competing for CPU and memory. Separating the benchmark runs and treating the first run as a warm-up (cold-cache artefact) produced more stable and comparable results. This taught us that performance evaluation must control for confounding factors to produce valid conclusions.

Synthetic datasets simplify reproducibility but limit generalisability. The Kaggle fraud detection dataset is synthetically generated, which made it freely available and easy to import. However, synthetic data may not capture the full distributional complexity of real-world transaction fraud. Future projects should consider using real-world datasets with appropriate anonymisation to test how MongoDB performs under more realistic access patterns.

NoSQL flexibility is both a strength and a responsibility. MongoDB imported the CSVs without any schema definition, inferring types from the data automatically. While this accelerated the data loading phase, it also meant that `is_fraud` was stored as an integer and not a boolean, requiring queries to match against `1` and `0` rather than `true` and `false`. Schema awareness remains important in MongoDB even though schema enforcement is optional.

4 Division of Group Members' Roles

Table 9: Group task distribution

Member	**Contribution**
Koay Chun Keat (164592)	– Set up the stand-alone MongoDB environment on Ubuntu Linux and Windows. – Wrote the import, index creation, and benchmark scripts. – Designed and tested all four aggregation pipeline queries. – Collected and analysed benchmark results.
Lai Yicheng (163729)	– Set up the Docker-containerised MongoDB environment. – Configured `docker-compose.yml` and volume mounts. – Executed data import and index creation for the Docker deployment. – Captured execution evidence and verified query output correctness.
Jacky Chung Sze Yung (163326)	– Prepared the report structure and wrote the result discussion. – Compiled the comparison and observation sections. – Organised screenshots and verified report formatting. – Prepared the presentation slides and recorded the demo video.
Overall Collaboration	– All members contributed to dataset selection, query design review, and final report proofreading. – All members reviewed the benchmark methodology and agreed on the interpretation of results.

5 Conclusion

This project evaluated MongoDB Community 8.0 in two deployment configurations — stand-alone installation on Ubuntu Linux and Docker containerisation — using a credit card fraud detection dataset of approximately 480 MB containing 1.85 million transaction records. Four meaningful aggregation pipeline queries were designed to extract fraud analytics insights: fraud counts by category, top merchants by fraud frequency, average fraudulent transaction amounts by hour of day, and a statistical comparison between fraudulent and legitimate transaction amounts. The analysis found that fraud concentrates in online and grocery point-of-sale categories, is spread thinly across merchants, peaks in volume late at night but in value during daytime hours, and averages 7.8 times the value of legitimate transactions.

Benchmarking across five runs per query confirms that the stand-alone deployment is faster for computation-heavy queries, achieving average execution times of 26.0 ms (Q3) and 2386.8 ms (Q4) compared to 30.8 ms and 2736.0 ms for Docker — a 15–19% difference attributable to the Overlay2 filesystem overhead of containerisation. For lightweight index-covered queries (Q1 and Q2), both platforms perform comparably in the single-digit millisecond range. Both implementations produce identical query results, confirming deployment transparency at the query logic level.

From a practical standpoint, Docker is the preferred choice for teams that value portability, reproducibility, and simplified provisioning, accepting a modest performance trade-off on heavy analytical workloads. Stand-alone deployment is preferable in performance-sensitive production environments where direct hardware access is available. For the medium-scale fraud detection workload evaluated here, either platform is viable, and the choice should be driven by operational context rather than raw performance alone.

References

[1] Docker Inc. mongo — Docker Hub. Docker Hub, 2024. Accessed: 2025-05-01.

[2] Anil Khadka and Arun Kumar Manandhar. A comparative study of SQL and MongoDB database for big data application. In *Proceedings of the 2019 International Conference on Computing, Communication, Chemical, Materials and Electronics Engineering*, 2019.

[3] MongoDB Inc. *Install MongoDB Community Edition on Ubuntu*, 2024. Accessed: 2025-05-01.

[4] MongoDB Inc. *MongoDB Manual*, 2024. Accessed: 2025-05-01.

[5] Kartik Shenoy. Credit card transactions fraud detection dataset. Kaggle, 2020. Accessed: 2025-05-01.

[6] Ciprian-Octavian Truica, Florin Radulescu, Alexandru Boicea, and Ion Bucur. Performance evaluation for CRUD operations in asynchronously replicated document oriented database. *Proceedings of the International Conference on Control Systems and Computer Science*, pages 191–196, 2015.

A Group Members' Contributions

Table 10: Detailed contribution log

Member	Detailed Contributions
Koay Chun Keat (164592)	– Wrote `standalone/setup.sh` and `standalone/setup.ps1` for MongoDB installation. – Wrote `standalone/import.sh` and `standalone/import.ps1` for data ingestion. – Wrote `shared/create_indexes.js`, `shared/queries.js`, and `shared/benchmark.js`. – Wrote `standalone/benchmark.sh` and collected standalone benchmark results. – Contributed to Introduction and Project Content sections of the report.
Lai Yicheng (163729)	– Wrote `docker/docker-compose.yml` and configured volume mounts. – Wrote `docker/import.sh`, `docker/import.ps1`, `docker/benchmark.sh`, and `docker/benchmark.ps1`. – Executed Docker import and index creation; collected Docker benchmark results. – Verified query output correctness across both deployments. – Contributed to the Comparison and Observation sections of the report.
Jacky Chung Sze Yung (163326)	– Prepared the report LaTeX structure and formatting. – Wrote the Lesson Learned, Conclusion, and Abstract sections. – Organised screenshots and checked report consistency. – Prepared the presentation slide deck. – Recorded and edited the group demo video.